# CS3210: Operating Systems

## Lecture 3 — x86 Architecture Essentials

John Kim
Georgia Institute of Technology

January 22, 2026

## Contents

**1 Administrative Notes** **2**

**2 Goals and Motivation** **2**

**3 The Von Neumann Machine Model** **2**

**4 The Three-Stage Boot Protocol** **2**
4.1 Stage 1: BIOS or UEFI . . . 3
4.2 Stage 2: The Bootloader . . . 3
4.3 Stage 3: Kernel Handoff . . . 3

**5 The Reset Vector and First Instructions** **3**
5.1 Where Execution Begins . . . 3
5.2 BIOS ROM in High Memory . . . 4

**6 Physical Memory Layout** **4**
6.1 Real-Mode (20-bit) Map . . . 4
6.2 32-bit Protected Mode Map . . . 4
6.3 Memory-Mapped Device I/O . . . 5

**7 x86 CPU Modes** **5**
7.1 Real Mode . . . 5
7.2 Protected Mode . . . 6

**8 CPU Registers** **6**
8.1 General-Purpose Registers . . . 6
8.2 Segment Registers . . . 7
8.3 EFLAGS . . . 7
8.4 Control Registers . . . 8

**9 Memory Segmentation** **8**
9.1 Segmentation as a Virtual Memory Mechanism . . . 8
9.2 The Global Descriptor Table . . . 8

9.3 Linear Addresses . . . . . 8
9.4 Why Paging Superseded Segmentation . . . . . 9

**10 x86 Assembly: AT&T Syntax** **9**
10.1 Overview and Conventions . . . . . 9
10.2 Addressing Modes . . . . . 9
10.3 Instruction Classes . . . . . 10

**11 Port-Mapped I/O in Practice: `readsect()`** **11**

**12 x86_64 and Long Mode** **11**
12.1 x86_64 as an Extension of x86 . . . . . 11
12.2 The Transition to Long Mode . . . . . 12

**13 Using QEMU to Inspect Register State** **12**

**14 Summary** **12**

**15 References** **13**

# 1 Administrative Notes

Several deadlines are active this week. **Lab 0** (the xv6 orientation lab) is due **January 23, 2026**—tomorrow. Lab 0 is worth 2% of the final grade; its primary purpose is to ensure every student has a working xv6 build environment and can navigate the codebase before the heavier lab work begins. Office hours are now running on a regular schedule; consult Piazza for the current time-slot listing.

**Lab 1**—covering startup, CPU stacks, debugging, and physical memory management—is officially assigned on January 23 and **due February 6, 2026**. Students therefore have two full weeks, but given the estimated 30-hour workload and the autograder's daily submission cap, starting immediately after Lab 0 is submitted is strongly advised.

# 2 Goals and Motivation

The explicit goal of this lecture is a narrow one: learn *just enough* x86 architecture to read and understand the xv6 source code. The course does not aim to produce x86 assembly programmers; rather, it equips students to parse the low-level boot and kernel code that xv6 is built from. Concretely, the lecture targets four areas:

1. **CPU modes** — how the processor transitions from its initial 16-bit real mode through 32-bit protected mode and, on modern hardware, into 64-bit long mode.
2. **Memory modes** — how addresses are formed in each CPU mode, including the role of segments, page tables, and the physical address space layout.
3. **Registers** — the complete set of programmer-visible registers: general purpose, segment, flags, and control registers.
4. **I/O devices** — how the CPU communicates with peripherals via port-mapped I/O and memory-mapped I/O.

# 3 The Von Neumann Machine Model

All x86 systems are instances of the **Von Neumann architecture**: a processor that fetches instructions from a shared memory, decodes and executes them, and stores results back to memory or to registers. The critical implication of the Von Neumann model for OS study is that *the program counter (EIP on x86) fully determines what the CPU does next.* The operating system controls execution by controlling EIP—either by saving and restoring it during context switches, or by arranging that privileged instructions trap into the kernel. Everything else—process isolation, memory management, scheduling—ultimately reduces to controlling what values end up in EIP and which privilege level the CPU is running at when it fetches the next instruction.

# 4 The Three-Stage Boot Protocol

Every x86 machine follows a well-defined three-stage protocol to go from power-on to a running kernel. Understanding each stage is essential for reading xv6's boot code.

### 4.1 Stage 1: BIOS or UEFI

When power is applied, the CPU begins executing code stored in **ROM** (read-only memory) on the motherboard—historically the **BIOS** (Basic Input/Output System), and on modern machines the **UEFI** (Unified Extensible Firmware Interface). This firmware is responsible for *initialising the hardware*: it runs the POST (Power-On Self-Test), enumerates attached devices, and sets up the minimal memory controller configuration needed to access DRAM. Once hardware is ready, the BIOS hands control to the bootloader by loading the first sector (512 bytes) of the boot device into RAM at a fixed address and jumping to it.

### 4.2 Stage 2: The Bootloader

The **bootloader** is the piece of software that bridges the firmware and the kernel. Because the BIOS hands off in 16-bit real mode with almost none of the hardware configured, the bootloader has two responsibilities. First, it must transition the CPU into 32-bit (or 64-bit) protected mode so that the kernel can use the full address space. Second, it must locate the kernel binary on disk, load it into memory, and jump to its entry point.

In xv6 the bootloader is split across two files: `bootasm.S` (the assembly stub that handles the mode transition) and `bootmain.c` (the C code that reads the kernel ELF binary from disk). On real systems the bootloader is GRUB, SYSLINUX, or similar, but the two-phase structure is identical.

### 4.3 Stage 3: Kernel Handoff

Once the bootloader has loaded the kernel into memory and jumped to its entry point, control passes to the kernel forever. The bootloader is *not* a resident piece of software; it is overwritten by the kernel's BSS and stack as soon as the kernel needs that memory. The kernel's entry point (in xv6: `entry` in `entry.S`) sets up paging, enables virtual memory, and calls `main()` in C, at which point all further boot logic runs in a fully configured kernel environment.

## 5 The Reset Vector and First Instructions

### 5.1 Where Execution Begins

On every 80386 and later x86 processor, the CPU begins executing at a fixed address called the **reset vector**: `0xFFFFFFF0`. This is the top 16 bytes of the 32-bit address space. However, the processor is *not yet* in 32-bit mode when it first awakens—it starts in a special variant of 16-bit real mode with a peculiarity: the code segment register `CS` is initialised to `0xF000` and the instruction pointer `IP` to `0xFFF0`. In real mode the physical address is normally computed as $\mathrm{CS} \times 16 + \mathrm{IP}$, which would give `0xFFFF0`—only 16 bytes from the 1 MB real-mode ceiling. Yet the processor temporarily asserts the top 12 address lines high, making the effective *physical* address `0xFFFFFFF0`, reaching directly into the BIOS ROM at the top of the 4 GB space.

The *very first instruction* stored at the reset vector is always a far jump:

Listing 1: First instruction executed after reset

```
ljmp $0xF000, $0xE05B
```

This long jump does two things simultaneously: it loads `CS` with `0xF000` and `IP` with `0xE05B`, which drops the CPU back into ordinary real mode (the 12 high address lines are no longer asserted), and

redirects execution into the BIOS initialisation code that lives at physical address `0xFE05B` in low memory. From this point the BIOS runs its POST and device initialisation as described above.

### 5.2 BIOS ROM in High Memory

Why is BIOS ROM accessible at both the top of the 4 GB address space and the top of the 1 MB real-mode space? The answer is physical address decoding. The BIOS ROM chip is 128 KB, residing at physical addresses `0xFFFE0000`–`0xFFFFFFFF`. The motherboard's memory controller is programmed to also *shadow* (mirror) the BIOS ROM into the real-mode range `0xF0000`–`0xFFFFF`, so the same code is accessible from both addresses. Before the first `ljmp`, the processor's top 12 address lines are driven high by the reset circuitry, making the reset vector physically reach the top of the BIOS ROM; after the `ljmp`, normal 20-bit real-mode addressing takes over and the ROM is accessed through its low shadow at `0xF0000`. This design allows the same physical ROM to be used in both address windows without any firmware changes.

## 6 Physical Memory Layout

### 6.1 Real-Mode (20-bit) Map

Real mode supports only a 20-bit physical address space, giving a ceiling of 1 MB. That megabyte has a historical layout that every x86 system preserves for backward compatibility:

| Address Range | Size | Contents |
|---|---|---|
| `0x00000`–`0x003FF` | 1 KB | Interrupt Vector Table (IVT) |
| `0x00400`–`0x004FF` | 256 B | BIOS Data Area |
| `0x00500`–`0x7BFFF` | ∼480 KB | Free conventional memory |
| `0x07C00`–`0x07DFF` | 512 B | Bootloader (MBR, loaded here by BIOS) |
| `0x0A000`–`0x0BFFF` | 128 KB | VGA display memory |
| `0x0C000`–`0x0CFFF` | varies | Video/device expansion ROMs |
| `0x0F000`–`0x0FFFF` | 64 KB | System BIOS shadow |

The bootloader at `0x7C00` is the hardcoded loading address that the BIOS has used since the original IBM PC; every compliant BIOS places the MBR sector at this address before jumping to it. The IVT at `0x0000` is the real-mode equivalent of the interrupt descriptor table: 256 four-byte entries, each a `CS:IP` pair pointing to a BIOS interrupt handler.

### 6.2 32-bit Protected Mode Map

When the CPU enters 32-bit protected mode it can address 4 GB. The x86 address space has a layered structure at this point:

| Address Range | Region | Notes |
|---|---|---|
| `0x00000000`–`0x0009FFFF` | Low Memory | DRAM; includes legacy IVT/BIOS areas |
| `0x000A0000`–`0x000BFFFF` | VGA Display | Memory-mapped I/O |
| `0x000C0000`–`0x000EFFFF` | 16-bit expansion ROMs | Memory-mapped I/O |
| `0x000F0000`–`0x000FFFFF` | BIOS ROM shadow | Memory-mapped I/O |
| `0x00100000`–varies | Extended Memory | Main DRAM above 1 MB |
| varies–`0xFDFFFFFF` | Unused | Depends on installed RAM |
| `0xFE000000`–`0xFFFFFFFF` | 32-bit MMIO devices | Memory-mapped device registers |

The gap between the top of installed DRAM and `0xFE000000` is simply unused. Modern systems with more than ∼3 GB of RAM must use the 64-bit address extension (PAE or long mode) to avoid the device address range occupying memory that would otherwise be addressable by DRAM.

### 6.3 Memory-Mapped Device I/O

x86 provides two distinct mechanisms for communicating with hardware devices. The first is **port-mapped I/O**: the CPU has a separate 16-bit *I/O address space* of 1024 distinct ports, accessed using the special instructions `IN` and `OUT`. This address space is completely separate from the memory address space; a write to I/O port `0x1F3` does not conflict with a write to memory address `0x1F3`. The limitation is that only 1024 ports exist, which is far too few for modern systems with dozens of devices.

The second mechanism is **memory-mapped I/O (MMIO)**: device registers are placed at specific physical addresses in the normal memory space. The CPU reads and writes these addresses with ordinary `MOV` instructions; the memory controller detects that the address falls in a device range and routes the transaction to the appropriate device instead of to DRAM. MMIO looks like "magic memory" from the software's perspective—reads and writes can have *side effects* (a write to a device register might trigger an action, and a read might return a value that changes on every access), which is why MMIO regions must never be cached and compilers must not reorder accesses to them (hence `volatile` in C device drivers).

The VGA display region at `0xA0000`–`0xBFFFF` is a classic example: writing a byte to `0xB8000` (in text mode) places a character on the screen immediately without any special I/O instruction.

## 7 x86 CPU Modes

### 7.1 Real Mode

**Real mode** is the 16-bit operating mode that every x86 processor starts in. It is preserved purely for backward compatibility with software written for the original 8086 processor in the late 1970s. In real mode:

- All registers are 16 bits wide (or used as 16-bit values).
- Physical addresses are 20 bits, computed by the formula:

$$\mathrm{PA} = \mathrm{CS} \times 16 + \mathrm{IP} = (\mathrm{CS} \ll 4) + \mathrm{IP}$$

  This gives a maximum physical address of `0xFFFF0 + 0xFFFF = 0x10FFEF`, which wraps to 1 MB on a 20-bit address bus.

- No memory protection. Any process can read or write any address.
- Interrupts are handled via the **Interrupt Vector Table (IVT)** at physical address `0x0000`: 256 four-byte `CS:IP` pairs.
- The processor has no privilege levels; all code runs at the highest privilege.

Real mode is a minimal, unprotected environment. The OS cannot protect itself from user code, and virtual memory is impossible. The only purpose of real mode today is to run the bootloader long enough to configure and enter protected mode.

### 7.2 Protected Mode

**Protected mode** is the 32-bit operating mode that modern kernels run in. To enter protected mode, the bootloader must:

1. Disable interrupts (`cli`).
2. Zero out the data, extra, and stack segment registers (`DS`, `ES`, `SS`).
3. Load the **Global Descriptor Table** (GDT) using `lgdt`.
4. Set the `PE` (Protection Enable) bit in control register `CR0`.
5. Perform a far jump to flush the instruction pipeline and update `CS` with a protected-mode selector.

The `bootasm.S` sequence in xv6 does exactly this. Once in protected mode, the processor supports 32-bit registers, a 4 GB address space, hardware-enforced privilege levels (rings 0–3), and page-based virtual memory.

## 8 CPU Registers

### 8.1 General-Purpose Registers

The x86 processor has eight 32-bit **general-purpose registers (GPRs)**. Although "general-purpose" implies they can hold any value, each register has a *conventional role* that the ABI and most code follows:

| Register | Mnemonic | Conventional role |
|---|---|---|
| `EAX` | Accumulator | Arithmetic results, return values |
| `EBX` | Base | Base address for memory access |
| `ECX` | Counter | Loop counters, `REP` count |
| `EDX` | Data | I/O port address, multiply/divide overflow |
| `ESI` | Source Index | Source pointer for string operations |
| `EDI` | Destination Index | Destination pointer for string operations |
| `EBP` | Base Pointer | Frame pointer (base of current stack frame) |
| `ESP` | Stack Pointer | Top of the current call stack |

Each 32-bit register can also be accessed as a 16-bit sub-register (`AX`, `BX`, etc.) or as 8-bit sub-registers for the first four (`AL/AH`, `BL/BH`, `CL/CH`, `DL/DH`), where the suffix `L` selects the low byte and `H` the high byte. This naming hierarchy exists for backward compatibility with 8-bit and 16-bit code.

The **instruction pointer** `EIP` is not a general-purpose register in the usual sense—software cannot directly write to it with `MOV`—but it is the most important register in the machine. It holds the address of the *next instruction to fetch*. It changes implicitly with every instruction executed, and explicitly through jumps (`JMP`, `CALL`, `RET`, `IRET`) and exceptions.

## 8.2 Segment Registers

There are six 16-bit **segment registers**: `CS` (code segment), `DS` (data segment), `SS` (stack segment), and three extra segments `ES`, `FS`, and `GS`. In real mode these hold the upper 16 bits of a 20-bit base address (see the real-mode address formula). In protected mode they hold **selectors**—indices into the Global Descriptor Table (GDT)—rather than raw addresses. The CPU automatically looks up the descriptor in the GDT to find the segment base, limit, and access rights whenever a memory reference is made through that segment register. OS code in xv6 sets up flat segments spanning the entire 4 GB space so that segmentation is effectively bypassed, with all memory protection delegated to paging.

## 8.3 EFLAGS

The **EFLAGS** register is a 32-bit collection of single-bit flags that record the current state of the CPU and control its behaviour. The most important flags for OS programming are:

- **CF** (Carry Flag) — set when an arithmetic operation generates a carry out of the most-significant bit.
- **ZF** (Zero Flag) — set when the result of an operation is zero.
- **SF** (Sign Flag) — set when the result is negative (MSB = 1).
- **OF** (Overflow Flag) — set on signed arithmetic overflow.
- **PF** (Parity Flag) — set when the low byte of a result has an even number of 1-bits.
- **AF** (Auxiliary Carry) — carry out of bit 3; used for BCD arithmetic.
- **TF** (Trap Flag) — when set, the CPU fires a debug exception after every instruction (single-step mode, essential for debuggers).
- **IF** (Interrupt Enable Flag) — when set, the CPU responds to external (maskable) hardware interrupts. `cli` clears it; `sti` sets it. The bootloader immediately calls `cli` to prevent spurious interrupts during the real-to-protected-mode transition.
- **DF** (Direction Flag) — controls whether string instructions (`MOVSB`, `SCASB`, etc.) increment or decrement `ESI/EDI`.
- **IOPL** (I/O Privilege Level, bits 12–13) — the minimum privilege level required to execute `IN/OUT` instructions.
- **VM** (Virtual 8086 Mode) — enables V86 mode for running real-mode code in a protected-mode environment.
- **RF** (Resume Flag) — suppresses debug faults on the next instruction after a breakpoint.

### 8.4 Control Registers

The **control registers** (`CR0`–`CR4` and `CR8` on x86_64) govern the CPU's operating mode and memory translation behaviour.

- **CR0** — holds the `PE` (Protection Enable, bit 0) and `PG` (Paging Enable, bit 31) flags, among others. Setting `PE` enters protected mode; setting `PG` activates paging.
- **CR2** — the processor writes the faulting linear address here whenever a page fault occurs. The page-fault handler reads `CR2` to determine what address caused the fault.
- **CR3** — holds the **physical base address** of the current page directory (page table root). Loading a new value into `CR3` switches the active address space, which is how context switches change virtual memory mappings.
- **CR4** — enables extended features including PAE (Physical Address Extension for >4 GB DRAM) and SSE instructions.

## 9 Memory Segmentation

### 9.1 Segmentation as a Virtual Memory Mechanism

**Segmentation** is x86's original virtual memory mechanism, predating paging. The core idea is to divide memory into *named regions* (segments), each described by a base address, a length (limit), and a set of access-permission flags. When code accesses memory through a segment register, the hardware checks that the offset does not exceed the limit and that the access type (read, write, execute) is permitted by the descriptor. If either check fails, the CPU raises a general protection fault.

### 9.2 The Global Descriptor Table

In protected mode, segment registers do not hold base addresses directly. Instead they hold **selectors**: 16-bit values that index into the **Global Descriptor Table (GDT)**, a memory-resident array of 8-byte *segment descriptors*. Each descriptor encodes the segment's 32-bit base, 20-bit limit, granularity (byte or 4 KB page units), privilege level (DPL), and type (code, data, system). The CPU locates the GDT through the **GDTR** register, which is loaded with a 48-bit value (base + limit) by the `lgdt` instruction.

Two other descriptor tables exist. The **Local Descriptor Table (LDT)** was intended to hold per-process segment descriptors, but it is almost never used in modern OSes. The **Task State Segment (TSS)** is a special system descriptor that the CPU uses when transitioning from user mode to kernel mode via a trap or interrupt: it provides the kernel stack pointer (`ESP0`) to switch to. xv6 sets up one TSS per CPU.

### 9.3 Linear Addresses

After the segment hardware computes *base + offset* (and performs its limit checks), the result is called a **linear address**. If paging is disabled, a linear address equals the physical address. If paging is enabled, the linear address is further translated by the page table hardware into the final physical address. In modern OS design the segment bases are all set to zero so that linear address

= virtual address (the programmer's view), with all actual address translation and protection delegated to paging.

### 9.4 Why Paging Superseded Segmentation

Segmentation and paging can both provide isolation and multiplexing, but paging is universally preferred in modern systems for three reasons:

1. **Arbitrary granularity** — paging maps memory in fixed-size pages (4 KB), which can be placed anywhere in physical memory independently. Segmentation requires each segment to be *contiguous* in physical memory, which makes defragmentation difficult as segments grow.
2. **Per-page permissions** — every 4 KB page can have independent read, write, and execute permissions. Segment permissions apply coarsely to the whole segment.
3. **Uniform mechanism** — using only paging for protection simplifies the virtual memory subsystem considerably. x86_64 even eliminated most segmentation support: in 64-bit mode CS, DS, ES, and SS bases are forced to zero, and only FS and GS retain configurable bases (used by the kernel for per-CPU data and by user programs for thread-local storage).

## 10 x86 Assembly: AT&T Syntax

### 10.1 Overview and Conventions

The GNU assembler (GAS), which is used by GCC and therefore by xv6, emits **AT&T syntax**. This differs from the Intel/NASM syntax found in most textbooks, so it is worth learning the differences explicitly.

The three key conventions are:

1. **Operand order is reversed**: the source operand comes *first* and the destination comes *second*. Where Intel writes `mov eax, edx` meaning "move `eax` into `edx`", AT&T writes `movl %eax, %edx`.
2. **Register names are prefixed with %**: `%eax`, `%ebx`, `%cr0`, etc.
3. **Immediate values are prefixed with $**: `$0x123`, `$42`.
4. **Operation size is indicated by a suffix**: `b` (byte, 8-bit), `w` (word, 16-bit), `l` (long, 32-bit), `q` (quad, 64-bit). So `movl` moves 32 bits; `movb` moves 8 bits.

### 10.2 Addressing Modes

x86 supports several ways to specify a memory address, and the AT&T syntax for each maps directly to a C-language equivalent:

Listing 2: AT&T addressing modes with C-equivalent comments

```
movl %eax, %edx        // edx = eax;                      (register)
movl $0x123, %edx      // edx = 0x123;                    (immediate)
movl 0x123, %edx       // edx = *((int32_t *)0x123);      (direct)
movl (%ebx), %edx      // edx = *((int32_t *)ebx);        (indirect)
movl 4(%ebx), %edx     // edx = *((int32_t *)(ebx+4));    (displaced)
```

The **displaced** mode is the most common: it adds a constant displacement to a register to form the effective address, exactly like the C idiom `ptr->field` (where the field offset is the displacement). A more general form supports a base register, an index register, and a scale factor, as illustrated by the comparison table below.

| GAS memory operand | NASM equivalent |
|---|---|
| `100` | `[100]` |
| `%es:100` | `[es:100]` |
| `(%eax)` | `[eax]` |
| `(%eax,%ebx)` | `[eax+ebx]` |
| `(%ecx,%ebx,2)` | `[ecx+ebx*2]` |
| `(,%ebx,2)` | `[ebx*2]` |
| `-10(%eax)` | `[eax-10]` |
| `%ds:-10(%ebp)` | `[ds:ebp-10]` |

The general form in GAS is `disp(base, index, scale)`, computing disp + base + index × scale. This addressing mode directly supports accessing array elements (`scale` = element size) and struct fields (`disp` = field offset) without extra arithmetic instructions.

## 10.3 Instruction Classes

x86 groups its instructions into several classes. The most relevant for reading kernel code are:

- **Data movement**: `MOV`, `PUSH`, `POP` — transfer values between registers, immediate constants, and memory.
- **Arithmetic / logic**: `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SHL`, `SHR`, `TEST`, `CMP` — operate on registers and memory; set EFLAGS bits as a side effect.
- **Control flow**: `JMP`, `JZ`, `JNZ`, `CALL`, `RET` — change EIP. Conditional jumps (`JZ`, `JNZ`, etc.) branch based on EFLAGS bits set by the preceding arithmetic instruction.
- **String operations**: `REP MOVSB`, `REP STOSD` — copy or fill memory blocks using `ECX` as a counter, `ESI` as the source, and `EDI` as the destination. The kernel uses these for `memcpy` and `memset` implementations.
- **I/O**: `IN`, `OUT` — transfer bytes, words, or double-words between a general-purpose register and an I/O port. The port address is either in `DX` or encoded as an 8-bit immediate.
- **System**: `INT` (software interrupt), `IRET` (interrupt return), `LGDT`, `LIDT`, `CLI`, `STI`, `HLT` — instructions that manage the processor's operating mode, interrupt state, and descriptor tables. These are privileged: executing most of them in ring 3 raises a general protection fault.

The authoritative reference for the complete instruction set is **Intel Architecture Manual Volume 2**. For any instruction whose behaviour is ambiguous, this manual is the definitive source and should be consulted directly.

# 11 Port-Mapped I/O in Practice: `readsect()`

To make port-mapped I/O concrete, consider the `readsect()` function from xv6's `bootmain.c`. This function uses `outb()` wrapper calls (which compile to `OUT` instructions) to communicate with the ATA (IDE) disk controller over the legacy I/O port range `0x1F0`–`0x1F7`:

Listing 3: readsect() in xv6 bootmain.c — reading one 512-byte sector via port I/O

```
void readsect(void *dst, uint offset) {
    waitdisk();
    outb(0x1F2, 1);                      // count = 1 sector
    outb(0x1F3, offset);                 // LBA bits 0-7
    outb(0x1F4, offset >> 8);            // LBA bits 8-15
    outb(0x1F5, offset >> 16);           // LBA bits 16-23
    outb(0x1F6, (offset >> 24) | 0xE0); // LBA bits 24-27 + master/slave
    outb(0x1F7, 0x20);                   // cmd: READ SECTORS

    waitdisk();
    insl(0x1F0, dst, SECTSIZE/4);        // read data (4 bytes at a time)
}
```

Each `outb(port, value)` emits a single `OUT` instruction that places `value` in the byte at I/O port `port`. The disk controller decodes the port address and interprets the sequence of writes as a command: sector count, 28-bit LBA (Logical Block Address), and finally the read command byte `0x20`. The disk then fetches the sector from the platters and signals completion by setting a status bit, which `waitdisk()` polls. The data is then read back 32 bits at a time using `insl` (an `INS` instruction with `ECX` iterations), filling the destination buffer.

This function is a perfect illustration of why port-mapped I/O exists: the disk controller exposes a small set of control registers on well-known ports, and the driver talks to it purely through `IN`/`OUT`. The 1024-port ceiling is not a problem here because the ATA interface uses only 8 ports.

# 12 x86_64 and Long Mode

## 12.1 x86_64 as an Extension of x86

**x86_64** (also called AMD64 or Intel 64) is a backward-compatible extension of the 32-bit x86 architecture. Most of what was described in this lecture for 32-bit x86—the boot protocol, real mode, segment registers, EFLAGS, paging—remains relevant and is encountered in the x86_64 boot path. The key differences are:

- Registers are widened from 32 bits to **64 bits**: `EAX` becomes `RAX`, `EBX` becomes `RBX`, and so on.
- **Eight new general-purpose registers** are added: `R8`–`R15`.
- The virtual address space expands to 48 bits in practice (the processor supports 64-bit pointers but only the low 48 bits are used by current hardware), giving a **256 TB** addressable space.
- Segmentation is largely disabled: in 64-bit mode, `CS`, `DS`, `ES`, and `SS` segment bases are all forced to zero by the hardware. Only `FS` and `GS` retain configurable bases, used respectively for thread-local storage and per-CPU kernel data.

- The page table hierarchy gains a fourth level (the **PML4** — Page Map Level 4) to support the wider address space.

### 12.2 The Transition to Long Mode

On a modern system the boot sequence is: **16-bit real mode → 32-bit protected mode → 64-bit long mode**. The transition from protected mode to long mode requires the following steps in strict order:

1. **Disable paging** (clear the `PG` bit in `CR0`) if it was previously enabled.
2. **Enable PAE** (Physical Address Extension) by setting bit 5 (`PAE`) in `CR4`. PAE is required for long mode because the larger page table structures (64-bit entries) depend on it.
3. **Load CR3** with the physical address of the **PML4** page table.
4. **Set the `EFER.LME` flag**: write to the **Model Specific Register (MSR)** at address `0xC0000080` using the `WRMSR` instruction. The LME (Long Mode Enable) bit is bit 8 of this register.
5. **Re-enable paging** by setting the `PG` bit in `CR0`.

After step 5 the CPU is in **compatibility mode**: it is technically in long mode, but still executes 32-bit instructions. To enter true **64-bit long mode**, the active GDT code segment descriptor must have its **D/B bit** (bit 22 of the second 32-bit word) *cleared* and its **L bit** (bit 21 of the second 32-bit word) *set*. A far jump to a code selector that points to such a descriptor completes the transition, after which the CPU executes full 64-bit instructions and all 16 registers are available.

xv6 as presented in this course targets 32-bit protected mode and does not go through the long mode transition. However, understanding the full boot sequence is essential background for reading any real-world 64-bit boot code (e.g., the Linux boot protocol).

## 13 Using QEMU to Inspect Register State

QEMU's monitor provides a simple way to observe the x86 register state at any point during a virtual machine's execution. After starting xv6 with `make qemu`, press **Ctrl-x p** to switch to the QEMU monitor, then type:

Listing 4: QEMU monitor command to display all registers

```
info registers
```

This prints the current values of all visible registers: the eight GPRs, EIP, EFLAGS, the six segment registers (with their hidden descriptor-cache fields showing base, limit, and access rights), and the four control registers. This is an invaluable tool for debugging boot code: you can pause execution at the reset vector, at the `lgdt/cr0` instructions, or at the kernel entry point and inspect exactly what the CPU sees at each step.

## 14 Summary

- **Boot protocol**: Every x86 machine goes through three stages—BIOS/UEFI, bootloader, kernel—in strict order. The BIOS is ROM firmware; the bootloader transitions the CPU to protected mode and loads the kernel ELF binary.

- **Reset vector**: The CPU begins execution at `0xFFFFFFF0` via a hardware mechanism that asserts the top 12 address lines, immediately jumping to BIOS code at `0xF0000` in the 1 MB real-mode space.
- **Real mode**: 16-bit, no protection, PA = CS $\ll 4 +$ IP, 1 MB addressable, IVT at `0x0000`. Exists only for backward compatibility; the bootloader exits it as quickly as possible.
- **Memory layout**: Real mode has a fixed 1 MB layout (IVT, BIOS data, free RAM, VGA, expansion ROMs, System BIOS). Protected mode extends to 4 GB with DRAM, MMIO holes for legacy devices, and 32-bit MMIO at the top.
- **Registers**: Eight 32-bit GPRs (EAX–ESP) with conventional roles; EIP (instruction pointer); six segment registers; EFLAGS (status, control, and system flags); control registers CR0–CR4 governing mode and translation.
- **Segmentation**: Protected-mode memory model based on GDT descriptors; provides base/limit/permission for each segment; produces linear addresses fed into the pager. Largely bypassed in modern OSes (flat segments) in favour of paging.
- **Paging vs. segmentation**: Paging wins because it allows non-contiguous physical placement, per-page granular permissions, and a uniform protection model. x86_64 largely eliminates segment-based protection.
- **AT&T assembly**: GAS syntax uses `op src, dst` order; `%reg` for registers; `$imm` for immediates; `b/w/l/q` size suffixes. Addressing modes range from register through displaced and scaled-index.
- **Port-mapped I/O**: 1024-port separate address space accessed via `IN/OUT` instructions; sufficient for legacy devices like ATA disk.
- **Memory-mapped I/O**: Device registers placed in the physical address space; accessed with ordinary `MOV`; reads and writes have hardware side effects.
- **Long mode**: x86_64's 64-bit mode; reached via a 5-step transition (disable paging, set PAE, load CR3, set EFER.LME, enable paging) followed by a far jump to a GDT descriptor with the L bit set. xv6 uses 32-bit protected mode, but modern systems and Linux use long mode.

# 15 References


- Tumanov, A. *CS3210: Design of Operating Systems — Lecture 3 Slides.* Georgia Institute of Technology. January 22, 2026.
- Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference.* Available at `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html`
- OSDev Wiki. "X86-64: Segmentation in Long Mode." `http://wiki.osdev.org/X86-64#Segmentation_in_Long_Mode`
- Cox, R.; Kaashoek, F.; Morris, R. *xv6: a simple, Unix-like teaching operating system.* MIT CSAIL. `https://pdos.csail.mit.edu/6.828/2022/xv6.html`
- Silberschatz, A.; Galvin, P. B.; Gagne, G. *Operating System Concepts*, 10th ed. John Wiley & Sons. 2018.

## Personal Notes